

CURATOR

Cost-Aware Environment Selection for Agentic Reinforcement Learning Training of Large Language Model

Name	Roll No.
Nishant Shah	23166
Ansh Bajpai	23108
M. Ruthik Reddy	23129

9. Initial RL Formulation: State, Action, Reward, Policy and Objective

Curator combines an inner GRPO reinforcement-learning loop with an outer non-stationary bandit-based environment scheduler. The outer scheduler dynamically allocates a fixed compute budget across multiple RL training environments.

9.1 State Space (S_t)

The state represents the current condition of the training process and the available environments.

$$S_t = (\text{environment statuses}, \mu^{\wedge}_I, n_i, c_i, B_t, S_{\text{val}}, t)$$

where:

- $\hat{\mu}_i$ = estimated utility of environment i
- n_i = number of times environment i has been selected
- c_i = measured compute cost of environment i
- B_t = remaining compute budget
- $S_{\text{val},t}$ = latest held-out validation benchmark score

Each environment can be classified into one of five states:

State	Meaning
S1: Unexplored	Insufficient observations are available to estimate its usefulness.
S2: Learning	The environment provides useful learning signal and the model is improving.
S3: Saturated	The environment has become too easy and provides little additional learning signal.
S4: Too Hard	Most rollouts fail and provide insufficient learning signal.
S5: Unreliable	The proxy signal disagrees with the observed held-out benchmark improvement.

9.2 Action Space A_t

At every training round, Curator selects a **mixture of environments** rather than choosing only one environment.

$$A_t = w_t \in \Delta^{N-1}$$

where w_i represents the proportion of the next training batch allocated to environment i .

$$w_i \geq \frac{\epsilon}{N}, \sum_{i=1}^N w_i = 1$$

The exploration floor ϵ/N ensures that an environment is not permanently discarded and can be rediscovered if its usefulness increases later.

At S1 — Unexplored Environment (Insufficient Data):

The environment has not been sampled enough to estimate its usefulness. The agent should explore it while avoiding excessive compute expenditure.

- **A1: Initial Exploration** — Allocate a small but sufficient fraction of the training budget to the environment to collect initial learning-progress and cost observations.
- **A2: Moderate Exploration** — Increase the allocation temporarily to obtain more reliable estimates of the environment's learning signal and compute cost.
- **A3: Maintain Exploration** — Keep the current exploration allocation while waiting for additional observations before making a stronger allocation decision.

At S2 — Learning Environment (Useful Signal):

The environment is producing meaningful learning progress and appears to be contributing useful training signal. The agent should exploit the environment while continuing limited exploration of alternatives.

- **B1: Increase Allocation** — Increase the environment's mixture weight because it is currently providing useful learning progress.
- **B2: Maintain Allocation** — Keep the current allocation when the environment continues to provide consistent learning signal.
- **B3: Moderate Increase** — Slightly increase the allocation when the environment has high estimated utility, but uncertainty remains.

At S3 — Saturated Environment (Low Learning Progress):

The environment has become relatively easy for the model. Its success rate is high, but additional training produces little learning progress.

- **C1: Reduce Allocation** — Decrease the environment's mixture weight to avoid spending compute where additional learning signal is limited.
- **C2: Minimum Exploration** — Reduce the allocation to the exploration floor while retaining a small probability of selecting the environment.
- **C3: Reallocate Compute** — Transfer the released compute budget toward environments with higher estimated utility or greater learning progress.

At S4 — Too-Hard Environment (Insufficient Learning Signal):

The environment is currently too difficult for the model, resulting in consistently unsuccessful rollouts and limited useful learning signal.

- **D1: Reduce Allocation** — Decrease the environment's mixture weight to prevent inefficient use of compute.
- **D2: Minimum Exploration** — Keep only the minimum exploration allocation so that the environment can be reconsidered as the model improves.
- **D3: Shift to Learnable Environments** — Reallocate compute toward environments that currently provide stronger and more reliable learning signals.

At S5 — Unreliable Environment / Signal Mismatch:

The environment's inexpensive proxy signal does not agree with the observed held-out benchmark improvement. The agent should avoid making a strong allocation decision based only on the proxy.

- **E1: Reduce Allocation** — Temporarily decrease the environment's allocation until the discrepancy is investigated.
- **E2: Maintain and Re-evaluate** — Keep a small allocation while collecting additional observations to determine whether the mismatch is temporary.
- **E3: Trigger Calibration** — Use the periodic benchmark evaluation to recalibrate the relationship between the proxy reward and actual benchmark improvement.

At S6 — Budget Exhausted (Terminal):

The available compute budget has been consumed. No additional training allocation can be made.

- **F1: Stop Training** — Terminate the environment-selection and training process.
- **F2: Final Evaluation** — Evaluate the trained model on the untouched test set.

- **F3: Generate ROI Report** — Produce the final environment-level ROI estimates and allocation statistics.

At S7 — Target Benchmark Achieved (Terminal):

The predefined benchmark target has been reached before the complete compute budget is consumed.

- **G1: Stop Early** — Terminate training because the target performance has been achieved.
- **G2: Final Evaluation** — Evaluate the final model on the untouched test set.
- **G3: Generate ROI Report** — Report the compute consumed, benchmark improvement and environment-level ROI.

Action Summary

State	Primary Decision	Possible Actions
S1 — Unexplored	Explore	Initial Exploration, Moderate Exploration, Maintain Exploration
S2 — Learning	Exploit	Increase Allocation, Maintain Allocation, Moderate Increase
S3 — Saturated	Reduce	Reduce Allocation, Minimum Exploration, Reallocate Compute
S4 — Too Hard	Reduce	Reduce Allocation, Minimum Exploration, Shift to Learnable Environments
S5 — Unreliable	Re-evaluate	Reduce Allocation, Maintain and Re-evaluate, Trigger Calibration
S6 — Budget Exhausted	Terminate	Stop Training, Final Evaluation, ROI Report
S7 — Target Achieved	Terminate	Stop Early, Final Evaluation, ROI Report

9.3 Reward R_t

Curator uses a **two-speed reward mechanism** consisting of a frequent low-cost proxy signal and a periodic benchmark-calibrated signal.

Fast Proxy Reward

The proxy reward is calculated every training round:

$$\tilde{r}_i = \frac{\alpha LP_i + \beta SR_i}{c_i}$$

where:

- LP_i = learning progress of environment i
- SR_i = signal richness
- c_i = measured compute cost
- α, β = weighting coefficients

This provides frequent feedback while accounting for the compute required by each environment.

Periodic Calibrated Reward

Every K rounds, Curator evaluates the model on a held-out calibration benchmark:

$$r_i^{cal} = \frac{\Delta S \cdot share_i}{c_i}$$

where:

- ΔS = observed improvement in the held-out benchmark
- $share_i$ = fraction of compute allocated to environment i
- c_i = measured compute cost

The calibrated signal is used to adjust the proxy reward so that the scheduler's inexpensive online signal remains aligned with actual benchmark improvement.

9.4 Policy π

Curator uses **Discounted Upper Confidence Bound (Discounted UCB)** to estimate the current utility of each environment.

$$UCB_i = \hat{\mu}_i(\gamma) + c \sqrt{\frac{\log T}{n_i}}$$

where:

- $\hat{\mu}_i(\gamma)$ = discounted mean reward
- n_i = number of observations for environment i
- T = current training round

- c = exploration coefficient
- γ = discount factor

The UCB scores are converted into environment mixture weights using a temperature-controlled softmax:

$$w_i = (1 - \epsilon) \text{softmax}\left(\frac{UCB_i}{\tau}\right) + \frac{\epsilon}{N}$$

The discounting allows Curator to adapt when an environment's usefulness changes as the LLM learns.

9.5 Objective

The main objective is to maximise improvement on the held-out benchmark while respecting the available compute budget:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[S_{val}(\theta_T) - S_{val}(\theta_0)]$$

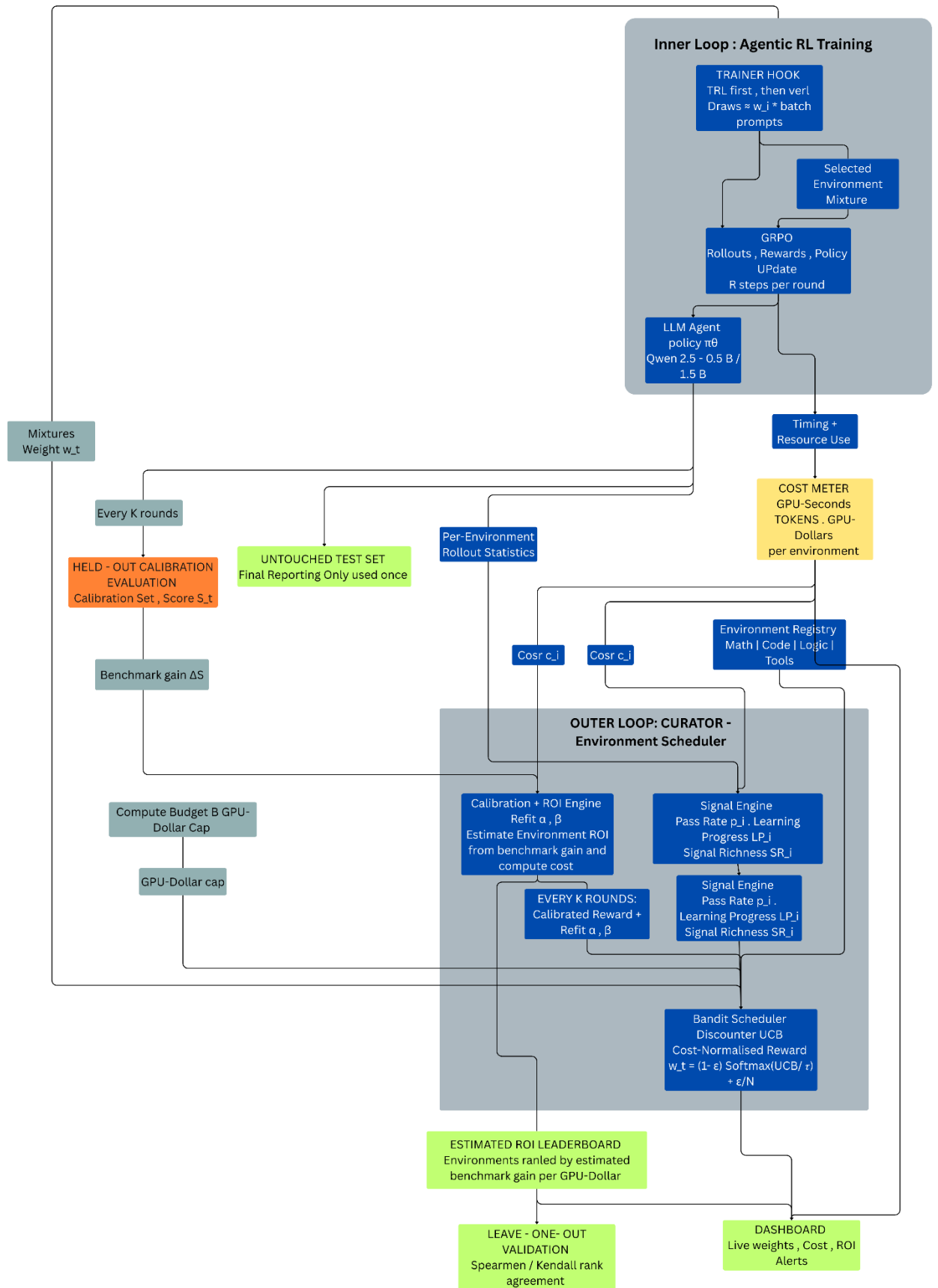
subject to:

$$\sum_t cost(w_t) \leq B$$

Thus, Curator aims to allocate limited GPU compute toward environments that provide the greatest **expected benchmark improvement per unit of compute**.

10. Architecture diagram

Canva link for the diagram: [Diagram Link](#)



11. DATASET / SIMULATION PLAN

The project will use multiple reasoning and agentic environments to study how an adaptive environment scheduler allocates limited training compute. The environments are selected to provide different levels of difficulty, learning progress, reward characteristics, and computational cost.

1. GSM8K – Mathematical Reasoning

GSM8K is a dataset of grade-school mathematical word problems used to evaluate mathematical reasoning. It provides a relatively learnable environment where the agent receives a reward based on whether the generated answer matches the expected answer.

<https://huggingface.co/datasets/openai/gsm8k>

2. MATH – Advanced Mathematical Reasoning

The MATH dataset contains more challenging competition-level mathematics problems. It will be used as a harder reasoning environment to study whether the Curator can identify environments that provide useful learning progress without spending excessive computation on environments that are too difficult.

<https://github.com/hendrycks/math>

3. Countdown – Structured Reasoning

Countdown is a procedural reasoning environment from Reasoning Gym where the agent must construct a valid mathematical expression using given numbers to reach a target value. It provides automatically verifiable rewards and can generate different levels of problem difficulty.

https://github.com/open-thought/reasoning-gym/tree/main/reasoning_gym/games

4. MBPP – Code Generation

The Mostly Basic Python Problems (MBPP) dataset contains Python programming problems that can be evaluated using automated tests. The number of passed tests provides a verifiable signal for measuring the learning progress of the language model.

<https://github.com/google-research/google-research/tree/master/mbpp>

5. Logic Reasoning Environments

Logic-based environments such as Knights and Knaves will be obtained through Reasoning Gym. These environments provide automatically verifiable reasoning tasks and allow the Curator to evaluate a different type of reasoning compared with mathematical and programming environments.

<https://github.com/EnvCommons/ReasoningGym>

6. Tool-Use / Function-Calling Environment

The Berkeley Function-Calling Leaderboard (BFCL) dataset will be considered for evaluating agentic tool-use behavior. The environment provides function-calling tasks where successful tool selection and argument generation can be evaluated automatically.

7. Synthetic Difficult and Noisy Environments

Additional environments will be created by modifying existing tasks to simulate specific conditions. These include **too-hard environments**, where the model receives little useful learning signal, and **noisy environments**, where the observed reward is less reliable. These environments will help test whether the Curator can identify environments that have poor training value.

8. Curator Simulation Environment

Before expensive LLM training, a synthetic simulator will be developed to model environment difficulty, learning progress, saturation, reward noise, computational cost, and transfer between environments. Since the optimal allocation is known in the simulator, it can be used to measure whether the Curator learns an efficient allocation strategy.

The experimental setup will therefore contain three stages: simulation, small-scale LLM training, and larger-scale validation. The final experiments will use a limited compute budget and evaluate whether adaptive allocation provides more benchmark improvement than static environment mixtures.

12. EVALUATION BASELINES

To rigorously evaluate the Curator, its performance will be compared with several environment-selection strategies. These baselines represent different ways of allocating the available training compute. The first baseline is Uniform Sampling. Every environment receives an equal amount of training compute regardless of its learning progress, difficulty, or computational cost. This provides a simple reference point for determining whether adaptive allocation provides an improvement. The second baseline is Static Mixture Training. A fixed mixture of environments is selected before training and remains unchanged throughout the experiment. This represents the conventional approach where the training distribution is manually specified and does not adapt to changes during learning.

The third baseline is Learning-Progress Based Selection. Environments are selected according to their observed learning progress. Environments showing larger improvements receive more training compute. This baseline tests whether the proposed cost-aware scheduler provides additional benefit beyond simply following learning progress. The fourth baseline is UCB-Based Environment Selection. A standard Upper Confidence Bound strategy is used to balance exploration and exploitation across environments. This allows the experiment to determine whether the additional cost normalization, proxy calibration, and ROI estimation used by Curator provide value beyond standard bandit selection.

Finally, the proposed Curator uses discounted UCB scores together with cost-normalized rewards, adaptive mixture weights, periodic held-out benchmark calibration, and environment-level ROI estimation.

The comparison will therefore examine whether the proposed method can achieve greater benchmark improvement for the same amount of training compute rather than simply training the model for a longer period.

13. KEY PERFORMANCE METRICS

The evaluation framework will measure both the effectiveness of the trained language model and the efficiency of the environment-selection strategy. The primary metric will be Held-Out Benchmark Improvement. The benchmark score will be measured before and after training on a held-out calibration set. The improvement in benchmark performance will indicate how effectively the allocated compute improves the model's reasoning or agentic capabilities.

The second metric will be Benchmark Improvement per GPU-Dollar. The benchmark improvement will be normalized by the measured computational cost of training. This measures whether the Curator can obtain more useful model improvement from the available compute budget.

$$\text{ROI} = \frac{\Delta S_{\text{benchmark}}}{\text{GPU-Dollar Cost}}$$

where $\Delta S_{\text{benchmark}}$ represents the improvement in the held-out benchmark score.

The third metric will be Environment Allocation Efficiency. This measures how the scheduler distributes training compute across environments over time. The experiment will examine whether environments showing useful learning progress receive more compute while environments that become saturated or provide weak signals receive less compute.

The fourth metric will be ROI Ranking Agreement. The estimated environment ROI produced by the Curator will be compared with the contribution measured using leave-one-out validation. Rank agreement can be measured using Spearman's rank correlation or Kendall's tau.

The fifth metric will be Learning Progress. For each environment, the change in task performance across training rounds will be measured. This provides the fast proxy signal used by the scheduler to estimate whether an environment is currently providing useful training information. The sixth metric will be Compute Cost. GPU time, GPU-seconds, generated tokens, and GPU-dollar equivalent cost will be recorded for each environment. This allows the system to explicitly account for environments that require substantially different amounts of computation.

Finally, the evaluation will measure Final Generalization Performance using an untouched test set. This test set will not be used by the scheduler for allocation decisions or ROI estimation and will be used only for final reporting.

The main evaluation will therefore answer three questions:

- Does adaptive environment selection improve the model?
- Does it use the available compute more efficiently?
- Can the Curator correctly identify which environments provide useful training value?

This gives the project a clear evaluation chain:

Environment Selection → Training → Benchmark Improvement → Compute Cost → ROI Estimation → Leave-One-Out Validation → Final Test Evaluation.

14. EXPECTED NOVELTY

The proposed Curator framework is expected to introduce the following contributions:

1. Environment-Level Adaptive Scheduling

Instead of selecting individual prompts or problem categories, Curator dynamically allocates training compute across different RL environments such as mathematics, coding, logic, and tool-use environments.

2. Compute-Aware Curriculum Selection

The scheduler considers both learning progress and the computational cost of each environment. Environments are therefore selected based on their estimated learning value per unit of compute, rather than learning progress alone.

3. Periodic Benchmark-Based Calibration

Curator combines fast proxy signals such as learning progress and signal richness with periodic evaluation on a held-out benchmark. This allows the relationship between proxy signals and actual benchmark improvement to be recalibrated during training.

4. Environment-Level ROI Estimation

The framework estimates the return obtained from allocating compute to each environment using benchmark improvement and measured GPU cost. This produces an environment ROI leaderboard that can be updated throughout training.

5. Validated Environment Attribution

The estimated ROI rankings will be validated using leave-one-out experiments, where individual environments are removed and the resulting change in final benchmark performance is measured. Rank agreement can be evaluated using Spearman or Kendall correlation.

6. Fixed-Budget Optimization

Rather than simply maximizing model performance by increasing training compute, Curator explicitly considers a fixed GPU/GPU-dollar budget and attempts to allocate that budget efficiently across environments.

7. Framework-Agnostic Integration Layer

Curator is designed as a scheduling layer that can operate on top of existing RL infrastructure such as TRL and potentially Verl, rather than requiring a replacement of the underlying GRPO training algorithm.

15. References

[1] A. Graves, M. G. Bellemare, J. Menick, R. Munos, and K. Kavukcuoglu, “Automated Curriculum Learning for Neural Networks,” *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 1311–1320, 2017.

[Link](#)

[2] T. Matiisen, A. Oliver, T. Cohen, and J. Schulman, “Teacher–Student Curriculum Learning,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 9, pp. 3732–3740, 2020.[Link](#)

[3] Z. Wang, G. Cui, K. Wan, and W. Zhao, “DUMP: Automated Distribution-Level Curriculum Learning for RL-based LLM Post-training,” *arXiv preprint arXiv:2504.09710*, 2025.[Link](#)

[4] X. Chen, J. Lu, M. Kim, D. Zhang, J. Tang, A. Piché, N. Gontier, Y. Bengio, and E. Kamaloo, “Self-Evolving Curriculum for LLM Reasoning,” *arXiv preprint arXiv:2505.14970*, 2025.

[Link](#)

[5] Z. Gao, J. Kim, W. Sun, T. Joachims, S. Wang, R. Y. Pang, and L. Tan, “Prompt Curriculum Learning for Efficient LLM Post-Training,” *International Conference on Learning Representations (ICLR)*, 2026.

[Link](#)

[6] Z. Gu et al., “Actor-Curator: Co-adaptive Curriculum Learning via Policy-Improvement Bandits for Scalable RL Post-training,” *arXiv preprint arXiv:2602.20532*, 2026.

<https://arxiv.org/abs/2602.20532>

[7] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems,” *arXiv preprint arXiv:2110.14168*, 2021.

<https://arxiv.org/abs/2110.14168>

[8] D. Hendrycks et al., “Measuring Mathematical Problem Solving With the MATH Dataset,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[Link](#)

[9] S. M. Kazemian et al., “Reasoning Gym: A Collection of Procedural Reasoning Environments for Reinforcement Learning,” 2025.

[Link](#)

[10] J. Austin et al., “Program Synthesis with Large Language Models,” *arXiv preprint arXiv:2108.07732*, 2021.[Link](#)

[11] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.

[Link](#)

[12] Hugging Face, “TRL: Transformer Reinforcement Learning,” GitHub repository.

[Link](#)

[14] ByteDance Seed, “verl: Volcano Engine Reinforcement Learning for LLMs,” GitHub repository.

[Link](#)

[15] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.

[Link](#)

[16] Z. Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.

[Link](#)